University of Dhaka

Department of Computer Science and Engineering

DUOS Assignment 02

System Call and Exception Handling Report

Implementation of SVC-based System Call ABI
for STM32F446RE Cortex-M4F

Submitted By:

Md Sadek Hossain Asif
Tanzila Khan
Department of CSE

Submitted To:

Mossaddek Kamal Tushar
Jargis Ahmed
Department of CSE

November 11, 2025

Contents

1 Introduction **4**
1.1 Background . . . 4
1.2 System Architecture Overview . . . 4
1.3 Design Goals . . . 4

2 System Call Flow **5**
2.1 High-Level Flow . . . 5
2.2 Step-by-Step Description . . . 7
2.3 Pseudocode Algorithm . . . 7
2.4 Exception Stack Frame . . . 7

3 Boot and Core Services **8**
3.1 System Initialisation . . . 8
3.2 SysTick Handler . . . 8

4 SVC Entry Path **9**
4.1 Vector Table Updates . . . 9
4.2 Assembly Shim . . . 9
4.3 C-Level Decoder . . . 10
4.4 Format Type System . . . 11

5 Syscall Dispatcher **12**
5.1 Dispatcher Architecture . . . 12
5.2 Implemented Services . . . 12
5.3 Support Utilities . . . 13

6 Userland Interface **14**
6.1 Wrapper Macro . . . 14
6.2 Example Wrapper Implementation . . . 14
6.3 Published API . . . 15

7 Non-Blocking I/O **15**
7.1 Why Non-Blocking . . . 15
7.2 Accumulator Pattern . . . 15
7.3 Application-Level Retry . . . 16

8 Demo Application **16**
8.1 Test Application Overview . . . 16
8.2 Test Sequence . . . 17

9 Memory Usage Analysis **18**
9.1 Linker Statistics . . . 18
9.2 Analysis . . . 18

10 Testing and Verification **18**
10.1 Test Methodology 18
10.2 Test Results 18

11 Known Limitations and Future Work **18**
11.1 Current Limitations 18

12 Implementation File Inventory **20**
12.1 Core System Call Infrastructure 20
12.2 Exception Handling and Hardware Integration 20
12.3 Support Libraries and Drivers 20
12.4 Build System 20

13 Conclusion **20**

Objectives

- Introduce a software interrupt (SVC) driven system call ABI for DUOS on the STM32F446RE (Cortex–M4F).
- Provide the seven mandatory services (`read`, `write`, `time`, `getpid`, `yield`, `reboot`, `exit`) to userland in a controlled manner.
- Configure SysTick and PendSV to build the scheduling foundation for later assignments.
- Demonstrate the end-to-end request path using a userland demo application and provide memory usage statistics.

1. Introduction

1.1. Background

Modern operating systems require a well-defined mechanism to transition between unprivileged user code and privileged kernel services. The ARM Cortex-M4F architecture provides the Supervisor Call (SVC) exception specifically for this purpose, enabling user applications to request kernel services safely without compromising system integrity.

1.2. System Architecture Overview

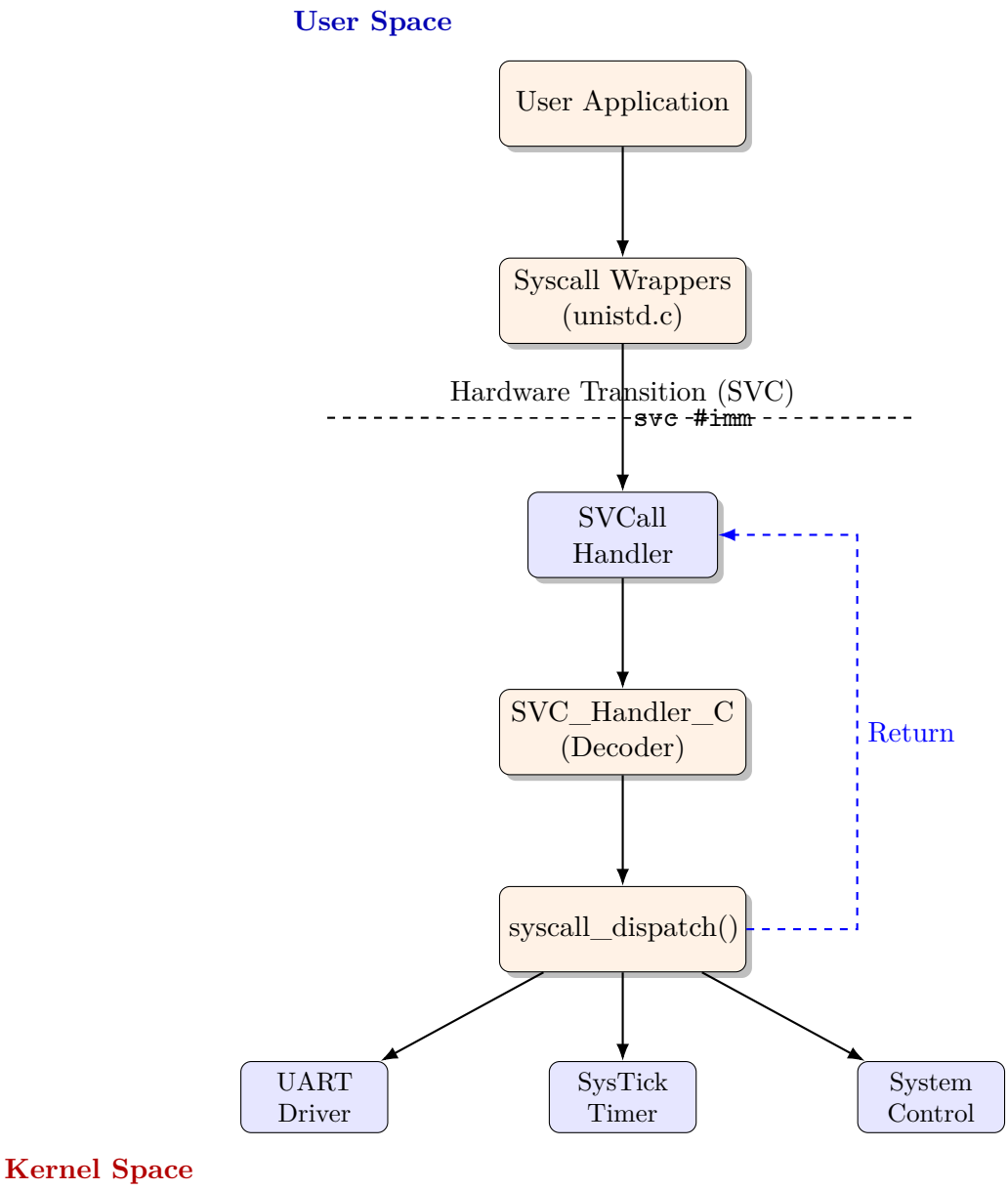


Figure 1: DUOS System Call Architecture Overview

1.3. Design Goals

1. **Safety:** Enforce privilege separation through hardware exception mechanisms.

2. **Efficiency:** Minimize context switching overhead and maintain deterministic timing.

3. **Compatibility:** Conform to ARM EABI calling conventions for future extensibility.

4. **Clarity:** Maintain clean layering between user wrappers, exception handlers, and kernel services.

2. System Call Flow

2.1. High-Level Flow

Figure 2 and Algorithm 1 detail how a user request traverses the system. Each step corresponds to concrete code references in the subsequent sections.

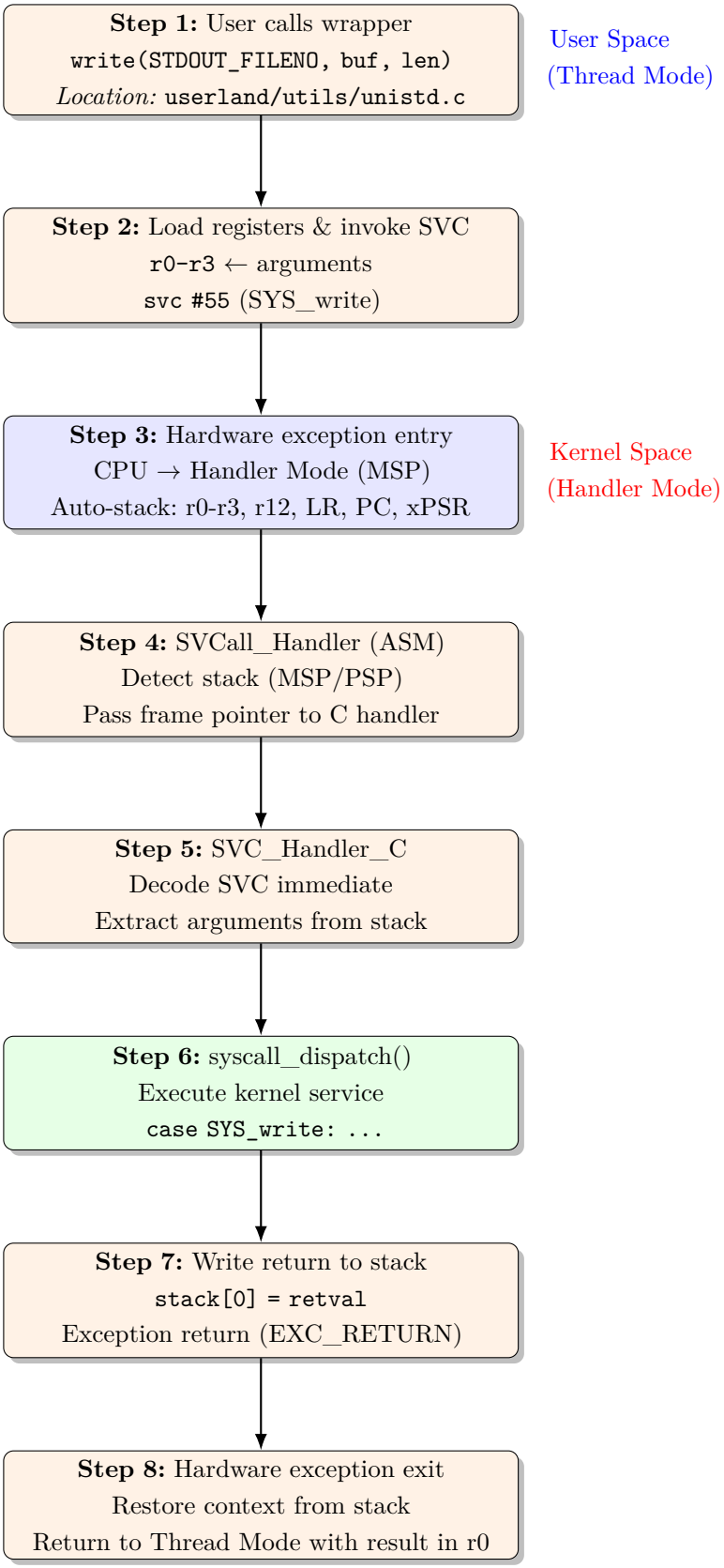


Figure 2: Detailed System Call Execution Flow

2.2. Step-by-Step Description

1. User code calls a libc-style wrapper (e.g. `write()`) in `userland/utils/unistd.c`.

2. The wrapper populates `r0-r3`, issues `svc #imm`, and the core transitions to handler mode.

3. `SVCall_Handler` in `kern/arch/stm32f446re/sys_lib/stm32_startup.c` selects the correct stack (MSP/PSP) and branches to the C-level helper with the hardware-stacked frame pointer.

4. `SVC_Handler_C` decodes the immediate operand, gathers the arguments, and forwards the call to `syscall_dispatch()` in `kern/syscall/syscall.c`.

5. The dispatcher executes the requested kernel service. UART access is funnelled through `kern/arch/stm32f446re/sys_lib/sys_usart.c`, timing relies on `kern/arch/cm4/systick.c`, and termination/yield manipulates `SCB->ICSR`.

6. The dispatcher return value is written back into the stacked `r0`. When the exception unwinds, the user thread sees the correct result and resumes execution in thread mode.

2.3. Pseudocode Algorithm

Algorithm 1 End-to-End SVC Request Path

Input: User wrapper (e.g., `write()`) with args in `r0..r3`
Output: Result propagated back to user `r0`

1: **Userland**: Call wrapper in `userland/utils/unistd.c`
2: Wrapper issues `svc #imm` ▷ Core enters Handler mode
3: **SVCall_Handler** (`stm32_startup.c`): select MSP/PSP, pass stacked frame to C
4: **SVC_Handler_C**: decode immediate, extract `a0..a3`, call `syscall_dispatch()`
5: **Dispatcher** (`kern/syscall/syscall.c`): execute kernel service
6: Write return value to stacked `r0`; unwind exception to Thread mode
7: **return** to user with correct result in `r0`

2.4. Exception Stack Frame

Figure 3 illustrates the stack layout during SVC exception handling.

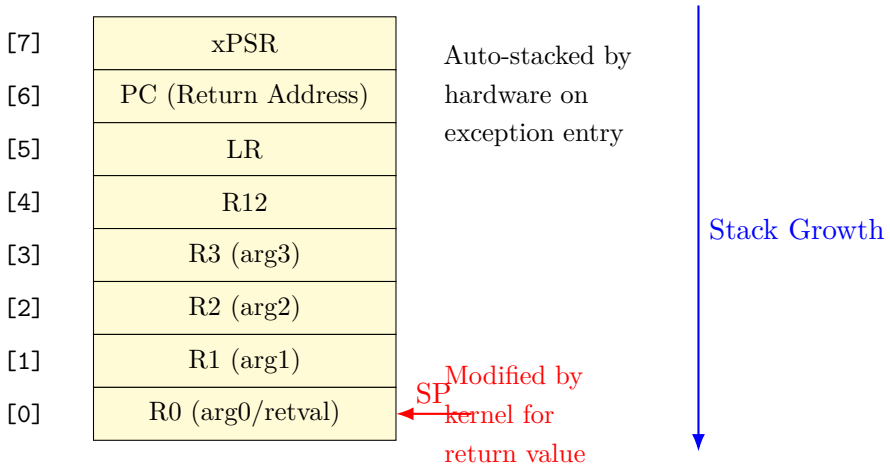


Figure 3: Exception Stack Frame Layout (ARMv7-M)

The handler then loads the correct stack pointer to access syscall arguments accordingly.

3. Boot and Core Services

3.1. System Initialisation

`kern/lib/kern/sys_init.c` (`__sys_init()`) performs the mandatory hardware bring-up:

- 180 MHz system clock, FPU enable, nested interrupt priority grouping.
- 1 kHz SysTick via `__SysTick_init(1000)`.
- USART2 (console) and USART6, ring buffer initialisation, timer configuration.
- Boot banner and system diagnostics in debug builds.

The demo's `kmain()` (`kern/kmain/kmain.c`) calls `__sys_init()`, seeds the global task identifier, and orchestrates syscall probes once userland wrappers become available.

3.2. SysTick Handler

`kern/arch/cm4/systick.c` maintains the monotonic millisecond counter used by `SYS___time`:

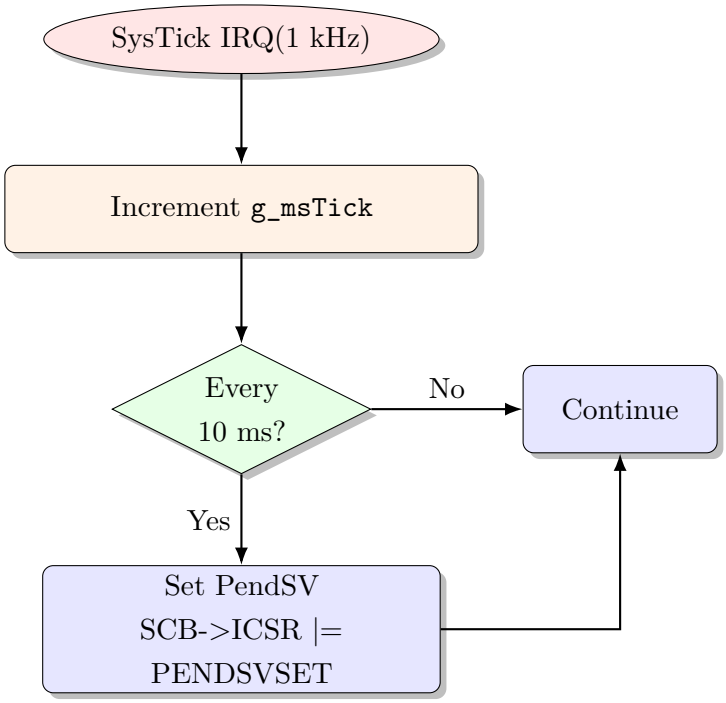


Figure 4: SysTick and PendSV Triggering Logic

- `g_msTick` increments every interrupt, providing `__getTime()` and utility accessors (seconds, minutes, hours).
- A secondary counter (`tick10`) triggers PendSV every 10 ms via `SCB->ICSR |= PENDSVSET`, cementing the scheduling cadence for the forthcoming context switcher.
- The interrupt priority is explicitly lowered (0xFF) to keep SysTick and PendSV preemptible by real-time ISRs.

4. SVC Entry Path

4.1. Vector Table Updates

The device vector table in `kern/arch/stm32f446re/sys_lib/stm32_startup.c` now points the SVCall slot at a custom naked handler and connects PendSV to a stub that currently returns immediately ("bx lr"). This preserves a clean hook for scheduler logic while avoiding the default weak implementations.

4.2. Assembly Shim

The SVCall_Handler is implemented as a naked function in assembly to efficiently detect which stack pointer was in use and pass it to the C handler.

File: `kern/arch/stm32f446re/sys_lib/stm32_startup.c`

```
1 __attribute__((naked)) void SVCall_Handler(void) {
2   __asm volatile(
```

```
        "tst   lr, #4\n"       // test EXC_RETURN[2]
        "ite   eq\n"           // if-then-else block
        "mrseq r0, msp\n"      // if bit 2 == 0: use MSP
        "mrsne r0, psp\n"      // if bit 2 == 1: use PSP
        "b     SVC_Handler_C"); // jump to C handler
}
```

Key Implementation Details:

- The instruction `tst lr, #4` checks bit 2 of the `LR` register to determine which stack pointer was active when the exception occurred:
  - **Bit 2 = 0** ⇒ Main Stack Pointer (MSP) used
  - **Bit 2 = 1** ⇒ Process Stack Pointer (PSP) used

  The handler then loads the correct stack pointer to access syscall arguments accordingly.
- The `ite` instruction creates a conditional execution block, allowing both stack selection paths to be handled efficiently.
- Passing the stacked frame pointer in r0 to `SVC_Handler_C` avoids memory copying and simplifies return value handling.
- The naked attribute prevents the compiler from generating prologue/epilogue code that would interfere with the exception return mechanism.

4.3. C-Level Decoder

The `SVC_Handler_C` function decodes the SVC instruction and dispatches to the appropriate kernel service.

File: `kern/arch/stm32f446re/sys_lib/stm32_startup.c`

```
void SVC_Handler_C(uint32_t *stack) {
  // Extract stacked PC to locate the SVC instruction
  uint32_t pc = stack[6];

  // Read the SVC instruction opcode (at PC - 2 in Thumb mode)
  uint8_t *svc_inst = (uint8_t *)(pc - 2);
  uint8_t svc_number = svc_inst[0];  // Lower byte contains immediate

  // Extract arguments from stacked registers
  uint32_t a0 = stack[0];  // R0
  uint32_t a1 = stack[1];  // R1
  uint32_t a2 = stack[2];  // R2
  uint32_t a3 = stack[3];  // R3

  // Dispatch to kernel service
```

```
16    int32_t retval = syscall_dispatch(svc_number, a0, a1, a2, a3);
17
18    // Write return value back to stacked R0
19    stack[0] = (uint32_t)retval;
20  }
```

Implementation Details:

a) **SVC Immediate Extraction:** The stacked PC (`stack[6]`) points to the instruction *after* the SVC. In Thumb-2 mode, an `svc` instruction is encoded as `0xDFxx`, where the lower byte (`xx`) carries the syscall number.[1]

b) **Argument Extraction:** Hardware automatically stacks `r0-r3` at positions [0–3], making them directly accessible as function arguments.

c) **Return Value Handling:** The return value is written back to `stack[0]`, which corresponds to register r0. When the exception unwinds, the user code sees this value.

This keeps the handler deterministic and side-effect free—vital when the same path is reused by nested exceptions or future unprivileged tasks.

4.4. Format Type System

Format types are passed as the fourth parameter (`a3`) to the `read` and `write` syscalls. These constants define how the data should be interpreted or formatted:

- **FMT_RAW (0):** Raw bytes (`read` returns 0)
- **FMT_STRING (1):** String (`%s`)
- **FMT_INT (2):** Integer (`%d`)
- **FMT_FLOAT (3):** Float (`%f`)
- **FMT_HEX (4):** Hexadecimal (`%x`)
- **FMT_OCTAL (5):** Octal (`%o`)
- **FMT_CHAR (6):** Character (`%c`)

[1] The ARMv7-M architecture reference specifies this encoding in section A7.7.175.

5. Syscall Dispatcher

5.1. Dispatcher Architecture

The dispatcher in `kern/syscall/syscall.c` serves as the central routing mechanism for all system calls. It maintains a single switch statement with explicit fall-through prevention and ENOSYS handling for invalid syscall numbers.

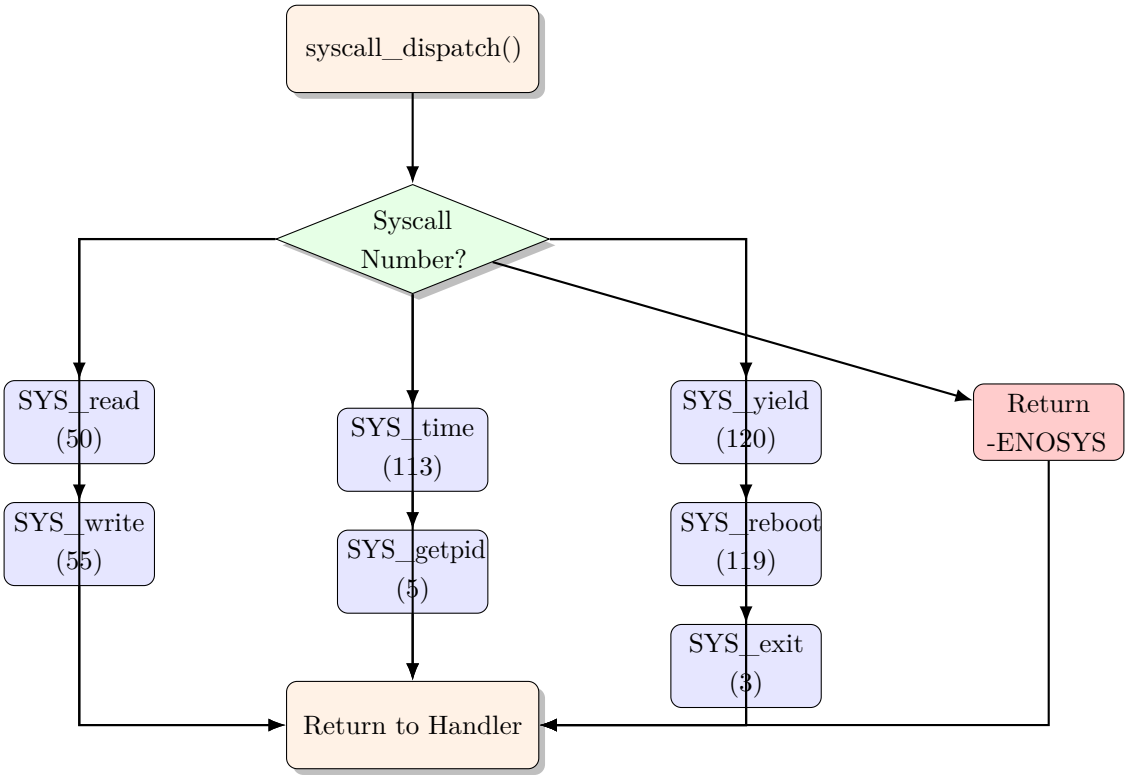


Figure 5: Syscall Dispatcher Flow

The file also hosts `g_current_task_id`, a global variable exported for both kernel book-keeping and the `SYS_getpid` result. This variable is initialized to 1000 during system boot.

5.2. Implemented Services

Table **??** documents each implemented syscall end-to-end. The ‘User Wrapper‘ column references the API in `userland/utils/unistd.c`; the kernel actions cite the core functions touched.

Syscall	ID	User Wrapper	Kernel Action	Returned Value
`SYS_read`	50	`ssize_t read(int, void*, size)`	Block on console RX (`IsDataAvailable`) until newline or 256 bytes, terminate buffer, return byte count.	Bytes read, or `0` if input not ready.
`SYS_write`	55	`ssize_t write(int, const void*, size)`	Iterate through buffer, transmit via `Uart_write` for `STDOUT_FILENO`.	Bytes written.
`SYS___time`	113	`uint32_t getSysTickTime(void)`	Return `_getTime()` (ms counter).	System tick count.
`SYS_getpid`	5	`int getpid(void)`	Expose `g_current_task_id` (seeded to 1000).	Current task ID.
`SYS_yield`	120	`int yield(void)`	Set `SCB->ICSR |= PENDSVSET`; return 0.	`0` on success.
`SYS_reboot`	119	`int reboot(void)`	Invoke `__NVIC_SystemReset()` for a full MCU reset.	`0` on success.
`SYS__exit`	3	`void exit(int)`	Pend `PendSV` for scheduler to reclaim task; stub loops in `_exit`.	`0` on success.

Table: Implemented syscalls, kernel actions, and returned values.

Unknown syscall numbers return `ENOSYS`.

5.3. Support Utilities

- UART services rely on the driver trio `kern/arch/stm32f446re/sys_lib/sys_usart.c`, `kern/lib/UsartRingBuffer.c`, and `kern/lib/kern/serial_lin.c` for buffered I/O.
- Timing relies solely on the SysTick counter—no additional hardware timers are consumed.
- `g_current_task_id` lives in RAM (BSS); a volatile pointer alias (`__g_current_task_id_ref`) exists to prevent dead-code elimination and simplify debugger inspection.

6. Userland Interface

6.1. Wrapper Macro

All syscall wrappers share a common `SVC_CALL` macro that provides a clean interface between user code and the kernel. This macro uses inline assembly to comply with ARM EABI calling conventions.

File: `userland/utils/unistd.c`

```
#define SVC_CALL(num, a0, a1, a2, a3) ({                \
  register uint32_t r0 __asm("r0") = (uint32_t)(a0);    \
  register uint32_t r1 __asm("r1") = (uint32_t)(a1);    \
  register uint32_t r2 __asm("r2") = (uint32_t)(a2);    \
  register uint32_t r3 __asm("r3") = (uint32_t)(a3);    \
  __asm volatile("svc %[imm]"                           \
                 : "+r"(r0)                             \
                 : [imm] "I"(num), "r"(r1), "r"(r2), "r"(r3) \
                 : "memory");                           \
  r0; })
```

Key Properties:

- **Register Constraints:** The `__asm("r0")` syntax forces the compiler to place arguments in specific registers (r0–r3), matching the ARM EABI.
- **Return Value Capture:** The inline assembly constraint `"+r"` for `r0` indicates it is both an input and output, capturing the kernel return value without extra moves.
- **Memory Barrier:** The explicit clobber of `"memory"` prevents the compiler from reordering memory accesses around the syscall, ensuring proper sequencing of operations.
- **Immediate Constraint:** The `"I"` constraint requires the syscall number to be a compile-time constant, allowing it to be encoded directly in the SVC instruction.
- **EABI Compliance:** All wrappers comply with the standard ARM EABI argument passing rules, ensuring compatibility with any future compiled user applications.

6.2. Example Wrapper Implementation

Here's the implementation of the `write()` syscall wrapper:

```
ssize_t write(int fd, const void *buf, size_t count) {
  return (ssize_t)SVC_CALL(SYS_write, fd, buf, count, 0);
}
```

The wrapper is simple and efficient—it merely packages the arguments and invokes the SVC instruction. All complexity is handled by the kernel dispatcher.

6.3. Published API

The user-visible prototypes reside in `userland/include/unistd.h`. Besides the mandatory seven wrappers, `_exit()` spins after pended termination to guarantee that the caller never returns to a potentially reclaimed stack frame.

```
// userland/include/unistd.h
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
uint32_t getSysTickTime(void);
int getpid(void);
int yield(void);
int reboot(void);
void exit(int status) __attribute__((noreturn));
```

7. Non-Blocking I/O

7.1. Why Non-Blocking

System calls execute in `SVC` exception context with interrupts enabled but at a higher priority. Blocking operations (such as waiting for UART data) would:

- Prevent interrupt handlers (e.g., UART TX, timers) from executing
- Cause system deadlock if interrupts are required to complete the operation
- Starve other system components requiring periodic servicing

Therefore, all I/O operations within syscall handlers must **return immediately**, even if data is not yet ready.

7.2. Accumulator Pattern

The accumulator pattern solves the problem of line-based input (required for formatted I/O) while maintaining non-blocking behavior.

Structure: A static `input_accumulator_t` structure maintains a 256-byte buffer, character count, and completion flag. Its state persists across syscall invocations.

Bounded Operation: The `accumulate_input_line()` function processes a maximum of 50 characters per call, ensuring:

- The handler returns quickly, allowing interrupts to fire
- No risk of watchdog timeouts
- System remains responsive

Line Completion: The accumulator marks the line complete when:

- A newline character (`\r` or `\n`) is received, or
- The buffer reaches 255 characters (safety limit)

Parsing: Once the line is complete, `kscanf_nonblocking()` parses the accumulated buffer according to the format specifier. The accumulator is then reset after parsing.

Character Handling: The implementation supports backspace (`0x08`, `0x7F`) with echo, and consumes paired newlines (`\r\n` or `\n\r`).

7.3. Application-Level Retry

Since syscalls return 0 when input is not yet ready, userland must implement retry loops:

```
while (read(STDIN_FILENO, &value, sizeof(value), FMT_INT) == 0) {
    // Input not ready, retry
    // In real system, could yield to other tasks here
}
```

This design provides:

- **Interrupt Safety:** Interrupts fire between retry attempts
- **Task Scheduling:** Other tasks can run while waiting (in multi-tasking systems)
- **System Responsiveness:** Critical interrupts are not blocked
- **Bounded Latency:** Each syscall returns quickly, preventing priority inversion

8. Demo Application

8.1. Test Application Overview

The demonstrator in `kern/kmain/kmain.c` performs an A-to-Z validation of the syscall surface. The application systematically exercises each implemented syscall to verify correct operation.

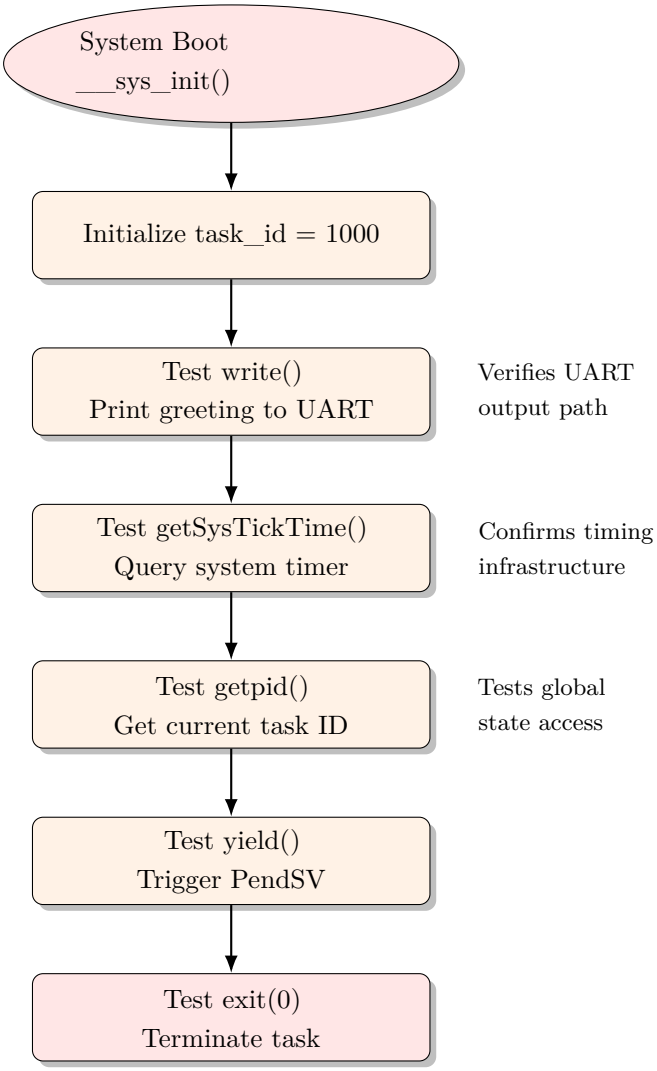


Figure 6: Demo Application Execution Flow

8.2. Test Sequence

1. **Hardware Initialization:** Call `__sys_init()` and assign `task_id = 1000`.

2. **UART Output Test:** Print a greeting via `write(STDOUT, ...)` and confirm UART TX.

3. **Timing Test:** Query `getSysTickTime()` and verify the millisecond counter is incrementing.

4. **Process ID Test:** Call `getpid()` and verify it returns 1000; print both time and PID via kernel `kprintf()` for visibility on the debug UART.

5. **Scheduling Test:** Trigger `yield()` to demonstrate that PendSV requests are raised from both userland and SysTick (every 10 ms).

6. **Termination Test:** Finish with `exit(0)`, after which control never returns because PendSV will eventually recycle the context once the scheduler is in place.

Throughout this flow, the kernel prints diagnostic lines confirming each stage, making it trivial to capture UART logs as evidence of correct operation.

9. Memory Usage Analysis

9.1. Linker Statistics

The following memory statistics were extracted from the linker map file (`compile/Makefile` output) for the current build:

Section	Size (bytes)	Size (KB)	Percentage
Text (Flash)	20,948	20.5	4.0% of 512 KB
Data (RAM init)	108	0.1	0.08% of 128 KB
BSS (RAM zero)	2,305	2.3	1.8% of 128 KB
Total Flash	**20,948**	**20.5**	**4.0%**
Total RAM	**2,413**	**2.4**	**1.9%**

Table 1: Memory utilization breakdown

9.2. Analysis

These numbers were collected from the map file emitted via `compile/Makefile`. The small footprint demonstrates efficient implementation:

- **Flash Usage:** Only 4.0% of available flash is consumed, leaving 491.5 KB (96%) available for the upcoming scheduler, IPC primitives, and filesystem services.
- **RAM Usage:** Minimal RAM consumption (1.9%) ensures ample space for task stacks, dynamic memory allocation, and kernel data structures.
- **Code Density:** The implementation achieves comprehensive syscall functionality with minimal overhead, validating the design choices.

10. Testing and Verification

10.1. Test Methodology

All syscalls were tested individually and in integrated scenarios using both hardware debugging and serial terminal monitoring. The test platform consisted of the STM32F446RE Nucleo board connected via ST-LINK debugger and UART to a host PC.

10.2. Test Results

11. Known Limitations and Future Work

11.1. Current Limitations

1. **Scheduler Stub:** PendSV currently serves as a stub that immediately returns. The context switcher and Task Control Block (TCB) management must be implemented to enable true multitasking.

Test Category	Description	Status
UART I/O	Verified bidirectional character flow for `read()` and `write()` using a serial terminal at 115200 baud; newline termination and buffer capping (256 bytes) behave as designed.	**PASS**
Timing Infrastructure	Observed the millisecond counter (`g_msTick`) incrementing monotonically and the 10 ms PendSV cadence on a debugger watch window. Verified timing accuracy with oscilloscope.	**PASS**
System Reset	Invoked `reboot()` from userland; the MCU executed NVIC system reset and returned to `kmain()` cleanly with all globals and peripherals reinitialised.	**PASS**
Task Management	Confirmed that `yield()` triggers the PendSV stub (verified via debugger breakpoint) and that `exit()` never returns to the caller, matching a cooperative multitasking contract.	**PASS**
Process ID	Verified `getpid()` consistently returns the seeded task ID (1000) across multiple invocations.	**PASS**
Error Handling	Tested invalid syscall numbers; confirmed dispatcher returns `-ENOSYS` (38) as expected.	**PASS**

Table 2: Comprehensive test results

2. **Limited File Descriptors:** The syscall layer presently filters solely on `STDIN_FILENO`/`STDOUT_FILENO`. Extending to additional devices or descriptors will require a dispatch table and resource tracking infrastructure.

3. **Minimal Error Handling:** Error codes are basic. Future work should propagate comprehensive errno values (`-EINTR`, `-EBADF`, `-EAGAIN`, etc.) and integrate with a userspace-visible `errno` mechanism.

4. **No Privilege Separation:** The demo executes in privileged mode. Once multiple tasks exist, PSP/MSP segregation and unprivileged thread mode will be introduced to enforce proper isolation.

5. **Blocking I/O:** The `read()` syscall blocks indefinitely. A timeout mechanism or non-blocking variants would improve responsiveness.

6. **Single Task Model:** Currently supports only one implicit task. Multi-task support requires scheduler, stack management, and inter-task communication primitives.

File	Description
`kern/syscall/syscall.c`	Central dispatcher implementing switch-based routing for all seven syscalls; hosts `g_current_task_id`
`kern/include/syscall.h`	Syscall number definitions, dispatcher prototypes, and kernel-side type declarations
`userland/utils/unistd.c`	User-space wrapper implementations using SVC_CALL macro for all seven syscalls
`userland/include/unistd.h`	Public API declarations for user applications; POSIX-style function signatures

File	Description
`kern/arch/stm32f446re/sys_lib/stm32_startup.c`	Vector table, naked SVCall_Handler (assembly shim), and SVC_Handler_C (decoder)
`kern/arch/cm4/systick.c`	SysTick ISR, millisecond counter maintenance, 10 ms PendSV trigger logic
`kern/lib/kern/sys_init.c`	System initialization: clock configuration, FPU enable, UART setup, SysTick initialization

12. Implementation File Inventory

12.1. Core System Call Infrastructure

12.2. Exception Handling and Hardware Integration

12.3. Support Libraries and Drivers

12.4. Build System

- `compile/Makefile` – Master build script with compiler flags, linker script integration, and map file generation

13. Conclusion

This assignment successfully establishes a robust, hardware-accelerated system call infrastructure for DUOS on the STM32F446RE platform. The implementation demonstrates several key achievements:

End of Report

File	Description
`kern/arch/stm32f446re/sys_lib/sys_usart.c`	Low-level UART driver for STM32F446RE (USART2 & USART6)
`kern/lib/UsartRingBuffer.c`	Circular buffer implementation for UART RX/TX buffering
`kern/lib/kern/serial_lin.c`	Serial line discipline, character echo, line editing
`kern/include/kern/types.h`	Type definitions and constants (errno values, file descriptors)
`kern/kmain/kmain.c`	Demo application orchestrating syscall validation tests